

SQL CREATE

Ing. Ladislav Körösi, PhD.
ÚRK, FEI STU

ladislav.korosi@stuba.sk

CREATE DATABASE

The CREATE DATABASE statement is used to create a database.

SQL CREATE DATABASE Syntax

```
CREATE DATABASE dbname;
```

Example

```
CREATE DATABASE test;
```

CREATE DATABASE - character set + collation

Full syntax:

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS]  
db_name  
[create_specification] ...
```

create_specification:

```
[DEFAULT] CHARACTER SET [=] charset_name  
| [DEFAULT] COLLATE [=] collation_name
```

Example:

```
CREATE DATABASE IF NOT EXISTS `test`  
DEFAULT CHARACTER SET latin1  
COLLATE latin1_swedish_ci;
```

CREATE DATABASE - character set + collation

The keyword `SCHEMA` can be used instead of `DATABASE`.

In MySQL, physically, a schema is synonymous with a database. You can substitute the keyword `SCHEMA` instead of `DATABASE` in MySQL SQL syntax, for example using `CREATE SCHEMA` instead of `CREATE DATABASE`.

Some other database products draw a distinction. For example, in the Oracle Database product, a schema represents only a part of a database: the tables and other objects owned by a single user.

CREATE DATABASE - character set + collation

All database options are stored in a text file named `db.opt` that can be found in the database directory.

The `CHARACTER SET` and `COLLATE` clauses make it possible to create databases with different character sets and collations on the same MySQL server.

CREATE DATABASE - character set + collation

MySQL chooses the database character set and database collation in the following manner:

- If both CHARACTER SET X and COLLATE Y are specified, character set X and collation Y are used.
- If CHARACTER SET X is specified without COLLATE, character set X and its default collation are used. To see the default collation for each character set, use the SHOW COLLATION statement.
- If COLLATE Y is specified without CHARACTER SET, the character set associated with Y and collation Y are used.
- Otherwise, the server character set and server collation are used.

CREATE DATABASE - character set + collation

SQL: SHOW COLLATION

Collation	Charset	Id	Default	Compiled	Sortlen
big5_chinese_ci	big5	1	Yes	Yes	1
big5_bin	big5	84		Yes	1
dec8_swedish_ci	dec8	3	Yes	Yes	1
dec8_bin	dec8	69		Yes	1
cp850_general_ci	cp850	4	Yes	Yes	1
cp850_bin	cp850	80		Yes	1
hp8_english_ci	hp8	6	Yes	Yes	1
hp8_bin	hp8	72		Yes	1
koi8r_general_ci	koi8r	7	Yes	Yes	1
koi8r_bin	koi8r	74		Yes	1
latin1_german1_ci	latin1	5		Yes	1

etc.

USE DATABASE

When you have multiple databases in your SQL Schema, then before starting your operation, you would need to select a database where all the operations would be performed. The SQL USE statement is used to select any existing database in SQL schema.

SQL USE Syntax

```
USE dbname;
```

Example

```
USE test;
```

CREATE TABLE

The CREATE TABLE statement is used to create a table in a database. Tables are organized into rows and columns; and each table must have a name.

SQL CREATE TABLE Syntax

```
CREATE TABLE table_name  
(  
  column_name1 data_type(size),  
  column_name2 data_type(size),  
  column_name3 data_type(size),  
  ....  
);
```

CREATE TABLE

The `column_name` parameters specify the names of the columns of the table.

The `data_type` parameter specifies what type of data the column can hold (e.g. `varchar`, `integer`, `decimal`, `date`, etc.).

The `size` parameter specifies the maximum length of the column of the table.

CREATE TABLE

Example:

```
CREATE TABLE `filmy`  
(  
  `ID` int(11),  
  `NAZOV` char(50),  
  `ROK` year(4),  
  `ZANER` char(20),  
  `IMDB` decimal(2,1)  
);
```

CREATE TABLE - character set + collation

Every table has a table character set and a table collation. The CREATE TABLE and ALTER TABLE statements have optional clauses for specifying the table character set and collation:

```
CREATE TABLE tbl_name (column_list)
[[DEFAULT] CHARACTER SET charset_name]
[COLLATE collation_name]
```

```
ALTER TABLE tbl_name
[[DEFAULT] CHARACTER SET charset_name]
[COLLATE collation_name]
```

CREATE TABLE - character set + collation

MySQL chooses the table character set and collation in the following manner:

- If both CHARACTER SET X and COLLATE Y are specified, character set X and collation Y are used.
- If CHARACTER SET X is specified without COLLATE, character set X and its default collation are used. To see the default collation for each character set, use the SHOW COLLATION statement.
- If COLLATE Y is specified without CHARACTER SET, the character set associated with Y and collation Y are used.
- Otherwise, the database character set and collation are used.

CREATE TABLE - character set + collation

Example:

```
CREATE TABLE `filmy`  
(  
  `ID` int(11),  
  `NAZOV` char(50),  
  `ROK` year(4),  
  `ZANER` char(20),  
  `IMDB` decimal(2,1)  
) CHARACTER SET utf8 COLLATE utf8_slovak_ci;
```

CREATE TABLE - character set + collation

Example:

```
CREATE TABLE `filmy`  
(  
  `ID` int(11),  
  `NAZOV` char(50) CHARACTER SET utf8 COLLATE  
utf8_slovak_ci,  
  `ROK` year(4),  
  `ZANER` char(20) CHARACTER SET utf8 COLLATE  
utf8_slovak_ci,  
  `IMDB` decimal(2,1)  
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

CREATE TABLE - ENGINE

MySQL supports several storage engines that act as handlers for different table types. MySQL storage engines include both those that handle transaction-safe tables and those that handle nontransaction-safe tables.

To determine which storage engines your server supports by using the `SHOW ENGINES` statement

CREATE TABLE - ENGINE

MySQL 5.1 supported storage engines:

- **MyISAM:** The default MySQL storage engine and the one that is used the most in Web, data warehousing, and other application environments. MyISAM is supported in all MySQL configurations, and is the default storage engine unless you have configured MySQL to use a different one by default.
- **Memory:** Stores all data in RAM for extremely fast access in environments that require quick lookups of reference and other like data. This engine was formerly known as the HEAP engine.

CREATE TABLE - ENGINE

MySQL 5.1 supported storage engines:

- **InnoDB:** A transaction-safe (ACID compliant) storage engine for MySQL that has commit, rollback, and crash-recovery capabilities to protect user data. InnoDB row-level locking (without escalation to coarser granularity locks) and Oracle-style consistent nonlocking reads increase multi-user concurrency and performance. InnoDB stores user data in clustered indexes to reduce I/O for common queries based on primary keys. To maintain data integrity, InnoDB also supports FOREIGN KEY referential-integrity constraints.

CREATE TABLE - ENGINE

MySQL 5.1 supported storage engines:

- **Merge:** Enables a MySQL DBA or developer to logically group a series of identical MyISAM tables and reference them as one object. Good for VLDB environments such as data warehousing.
- **Archive:** Provides the perfect solution for storing and retrieving large amounts of seldom-referenced historical, archived, or security audit information.
- **Federated:** Offers the ability to link separate MySQL servers to create one logical database from many physical servers. Very good for distributed or data mart environments.

CREATE TABLE - ENGINE

MySQL 5.1 supported storage engines:

- **NDB** (also known as NDBCLUSTER)—This clustered database engine is particularly suited for applications that require the highest possible degree of uptime and availability.
- **CSV**: The CSV storage engine stores data in text files using comma-separated values format. You can use the CSV engine to easily exchange data between other software and applications that can import and export in CSV format.

CREATE TABLE - ENGINE

MySQL 5.1 supported storage engines:

- **Blackhole:** The Blackhole storage engine accepts but does not store data and retrievals always return an empty set. The functionality can be used in distributed database design where data is automatically replicated, but not stored locally.
- **Example:** The Example storage engine is “stub” engine that does nothing. You can create tables with this engine, but no data can be stored in them or retrieved from them. The purpose of this engine is to serve as an example in the MySQL source code that illustrates how to begin writing new storage engines. As such, it is primarily of interest to developers.

CREATE TABLE + Constraints

SQL CREATE TABLE + CONSTRAINT Syntax

```
CREATE TABLE table_name  
(  
column_name1 data_type(size) constraint_name,  
column_name2 data_type(size) constraint_name,  
column_name3 data_type(size) constraint_name,  
....  
);
```

SQL Constraints

SQL constraints are used to specify rules for the data in a table.

If there is any violation between the constraint and the data action, the action is aborted by the constraint.

Constraints can be specified when the table is created (inside the CREATE TABLE statement) or after the table is created (inside the ALTER TABLE statement).

CREATE TABLE + Constraints

In SQL, we have the following constraints:

- **NOT NULL** - Indicates that a column cannot store NULL value,
- **UNIQUE** - Ensures that each row for a column must have a unique value,
- **PRIMARY KEY** - A combination of a NOT NULL and UNIQUE. Ensures that a column (or combination of two or more columns) have a unique identity which helps to find a particular record in a table more easily and quickly,
- **FOREIGN KEY** - Ensure the referential integrity of the data in one table to match values in another table,

CREATE TABLE + Constraints

In SQL, we have the following constraints:

- **CHECK** - Ensures that the value in a column meets a specific condition,
- **DEFAULT** - Specifies a default value for a column.

CREATE TABLE + Constraint NOT NULL

The NOT NULL constraint enforces a column to NOT accept NULL values.

The NOT NULL constraint enforces a field to always contain a value. This means that you cannot insert a new record, or update a record without adding a value to this field.

Example:

```
CREATE TABLE PersonsNotNull  
(P_Id int NOT NULL,  
  LastName varchar(255) NOT NULL,  
  FirstName varchar(255));
```

CREATE TABLE + Constraint NOT NULL

Example:

```
CREATE TABLE PersonsNotNull  
(P_Id int NOT NULL,  
LastName varchar(255) NOT NULL,  
FirstName varchar(255));
```

CREATE TABLE + Constraint NOT NULL

Example:

```
CREATE TABLE `filmy`  
(  
  `ID` int(11) NOT NULL,  
  `NAZOV` char(50) NOT NULL,  
  `ROK` year(4) NOT NULL,  
  `ZANER` char(20) NOT NULL,  
  `IMDB` decimal(2,1) NOT NULL  
);
```

CREATE TABLE + Constraint UNIQUE

The UNIQUE constraint uniquely identifies each record in a database table.

The UNIQUE and PRIMARY KEY constraints both provide a guarantee for uniqueness for a column or set of columns.

A PRIMARY KEY constraint automatically has a UNIQUE constraint defined on it.

Note that you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.

CREATE TABLE + Constraint UNIQUE

Example (MySQL):

```
CREATE TABLE Persons  
(P_Id int NOT NULL,  
LastName varchar(255) NOT NULL,  
FirstName varchar(255),  
UNIQUE (P_Id));
```

CREATE TABLE + Constraint UNIQUE

Example (MySQL):

```
CREATE TABLE Persons  
(P_Id int NOT NULL,  
LastName varchar(255) NOT NULL,  
FirstName varchar(255),  
CONSTRAINT uc_PersonID UNIQUE (P_Id, LastName));
```

CREATE TABLE + Constraint UNIQUE

Example:

```
CREATE TABLE `filmy`  
(  
  `ID` int(11) NOT NULL UNIQUE,  
  `NAZOV` char(50) NOT NULL,  
  `ROK` year(4) NOT NULL,  
  `ZANER` char(20) NOT NULL,  
  `IMDB` decimal(2,1) NOT NULL  
);
```

CREATE TABLE + Constraint PRIMARY KEY

The PRIMARY KEY constraint uniquely identifies each record in a database table.

Primary keys must contain UNIQUE values.

A primary key column cannot contain NULL values.

Most tables should have a primary key, and each table can have only ONE primary key.

CREATE TABLE + Constraint PRIMARY KEY

Example:

```
CREATE TABLE Persons
(  
  P_Id int NOT NULL,  
  LastName varchar(255) NOT NULL,  
  FirstName varchar(255),  
  Address varchar(255),  
  City varchar(255),  
  PRIMARY KEY (P_Id)  
)
```

CREATE TABLE + Constraint PRIMARY KEY

Example:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CONSTRAINT      pk_PersonID      PRIMARY      KEY
  (P_Id,LastName)
)
```

CREATE TABLE + Constraint PRIMARY KEY

Example:

```
CREATE TABLE `filmy`  
(  
  `ID` int(11) NOT NULL,  
  `NAZOV` char(50) NOT NULL,  
  `ROK` year(4) NOT NULL,  
  `ZANER` char(20) NOT NULL,  
  `IMDB` decimal(2,1) NOT NULL,  
  UNIQUE(ID),  
  PRIMARY KEY(ID)  
);
```

CREATE TABLE + Constraint FOREIGN KEY

A FOREIGN KEY in one table points to a PRIMARY KEY in another table.

Example:

TABLE PERSON

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

TABLE ORDERS

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	2
4	24562	1

CREATE TABLE + Constraint FOREIGN KEY

Note that the "P_Id" column in the "Orders" table points to the "P_Id" column in the "Persons" table.

The "P_Id" column in the "Persons" table is the PRIMARY KEY in the "Persons" table.

The "P_Id" column in the "Orders" table is a FOREIGN KEY in the "Orders" table.

The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.

CREATE TABLE + Constraint FOREIGN KEY

The FOREIGN KEY constraint also prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the table it points to.

The following SQL creates a FOREIGN KEY on the "P_Id" column when the "Orders" table is created:

CREATE TABLE + Constraint FOREIGN KEY

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
PRIMARY KEY (P_Id)
)
```

CREATE TABLE + Constraint FOREIGN KEY

```
CREATE TABLE Orders
(
  O_Id int NOT NULL,
  OrderNo int NOT NULL,
  P_Id int,
  PRIMARY KEY (O_Id),
  FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
)
```

CREATE TABLE + Constraint CHECK

The CHECK constraint is used to limit the value range that can be placed in a column.

If you define a CHECK constraint on a single column it allows only certain values for this column.

If you define a CHECK constraint on a table it can limit the values in certain columns based on values in other columns in the row.

CREATE TABLE + Constraint CHECK

The following SQL creates a CHECK constraint on the "P_Id" column when the "Persons" table is created. The CHECK constraint specifies that the column "P_Id" must only include integers greater than 0:

```
CREATE TABLE Persons  
(P_Id int NOT NULL,  
  LastName varchar(255) NOT NULL,  
  FirstName varchar(255),  
  Address varchar(255),  
  City varchar(255),  
  CHECK (P_Id>0))
```

CREATE TABLE + Constraint CHECK

Example:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CONSTRAINT chk_Person CHECK (P_Id>0)
)
```

CREATE TABLE + Constraint CHECK

Example:

```
CREATE TABLE Persons
(
  P_Id int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Address varchar(255),
  City varchar(255),
  CONSTRAINT chk_Person CHECK (P_Id>0 AND
  City='Sandnes')
)
```

CREATE TABLE + Constraint DEFAULT

The DEFAULT constraint is used to insert a default value into a column.

The default value will be added to all new records, if no other value is specified.

Example:

```
CREATE TABLE Persons  
(P_Id int NOT NULL,  
  LastName varchar(255) NOT NULL,  
  FirstName varchar(255),  
  Address varchar(255),  
  City varchar(255) DEFAULT 'Sandnes')
```

CREATE TABLE + Constraint DEFAULT

The DEFAULT constraint can also be used to insert system values, by using functions like GETDATE():

Example:

```
CREATE TABLE Orders
(
  O_Id int NOT NULL,
  OrderNo int NOT NULL,
  P_Id int,
  OrderDate date DEFAULT GETDATE()
)
```

CREATE INDEX

The `CREATE INDEX` statement is used to create indexes in tables.

Indexes allow the database application to find data fast; without reading the whole table.

An index can be created in a table to find data more quickly and efficiently.

The users cannot see the indexes, they are just used to speed up searches/queries.

CREATE INDEX

Note: Updating a table with indexes takes more time than updating a table without (because the indexes also need an update). So you should only create indexes on columns (and tables) that will be frequently searched against.

SQL CREATE INDEX Syntax

Creates an index on a table. Duplicate values are allowed:

```
CREATE INDEX index_name  
ON table_name (column_name)
```

CREATE INDEX

SQL CREATE UNIQUE INDEX Syntax

Creates a unique index on a table. Duplicate values are not allowed:

```
CREATE UNIQUE INDEX index_name  
ON table_name (column_name)
```

If you want to create an index on a combination of columns, you can list the column names within the parentheses, separated by commas:

```
CREATE INDEX PIndex  
ON Persons (LastName, FirstName)
```